

Lecture 17: Zero-Knowledge Interactive Proofs

Notes by Yael Kalai

MIT - 6.5620

Lecture 17 (November 5, 2025)

Warning: This document is a rough draft, so it may contain bugs. Please feel free to email me with corrections.

Outline

- Interactive proofs
- Zero-knowledge interactive proofs
- Commitment schemes from LWE

Recap

In the last two classes we learned about fully homomorphic encryption (FHE) schemes. FHE is an important primitive in today's world where we have large amounts of (possibly sensitive) data that we cannot store locally on our own devices. FHE allows us to store our data encrypted on an untrusted platform, and yet allow the platform to do computations on our encrypted data. Moreover, even if our sensitive data is stored on a trusted platform, for example our medical data is stored on our hospital servers, which we supposedly trust, the hospitals may want to collaborate and learn from their joint sensitive medical data. FHE allows them to collaborate without revealing to each other sensitive information about each other's data. FHE indeed allows us to obtain secrecy but what about integrity?

Suppose we store our data on an untrusted platform and then request the platform to perform computations on our encrypted data. How do we know that indeed the platform is doing the instructed computation? In other words, *can we efficiently verify that a computation was done correctly?* Namely, is there a *succinct* and *efficiently verifiable* proof that we can append to the output of a computation attesting to the fact that this output is indeed correct? This is the topic for the next five lectures.

Following our convention that "efficient" means polynomial time, we ask which computations (beyond P) have proofs of correctness that can be verified in polynomial time? This is precisely the definition of the complexity class NP, which is the set of all languages that

have membership proofs (aka witnesses) that can be checked by a polynomial-time verifier algorithm, denoted $\mathcal{V}$. We would like proofs of correctness for languages outside of NP. More precisely, our focus is on a fine-grained version of the question above. Namely, *do there exist proofs of correctness for time T computations that can be verified in time $\ll T$ (say time T^ϵ or even $\text{polylog}(T)$)?* Unfortunately, we do not believe that every T -computable language has a proof (or a “witness”) of size $\ll T$. As you learned by now, cryptography is an art of overcoming such barriers. We overcome this barrier by changing the definition of a proof and by making use of some cryptographic magic!

Interactive Proofs and Zero-Knowledge Proofs

Proof systems have been studied by mathematicians for thousands of years, starting from Euclid (300 BCE). A huge amount of progress in computer science has been made by attempting to computationally formalize the notion of a proof. This led to the definition of the complexity class NP, which is the set of all languages L such that membership in the language $x \in L$ can be efficiently verified given a polynomial-size proof $w \in \{0,1\}^{\text{poly}(n)}$ (called a witness).

Definition 1. A language $L \subseteq \{0,1\}^*$ is in NP if there exists a polynomial $m : \mathbb{N} \rightarrow \mathbb{N}$ and a poly-time algorithm R such that for every $n \in \mathbb{N}$ and every $x \in \{0,1\}^n$, $x \in L$ if and only if there exists $w \in \{0,1\}^{m(n)}$ such that $R(x, w) = 1$.

Until recently, all proof systems were of a somewhat similar form which is simply a list of formulas that follow from a set of inference rules and axioms.

This changed in the mid eighties when Goldwasser, Micali and Rackoff defined the notion of a *zero-knowledge proof* [2]. Intuitively, a zero-knowledge proof is one that reveals no information beyond the validity of the statement (and can be verified in polynomial time). What does “no information” mean? How is this formalized? Goldwasser et al. formalized it as follows: No information means we could have generated it on our own. However, with such formulation zero-knowledge proofs exist only for easy languages (i.e., ones that are in P).

To avoid this limitation, they completely changed the way we think about proofs. They defined a new notion called *interactive proofs*. Such proofs extend upon the classical notion of “proofs” in two ways. First, rather than solely considering a verifier algorithm $\mathcal{V}$, we instead think of the proof as arising from *interaction* between the verifier $\mathcal{V}$ and a prover algorithm $\mathcal{P}$. Second, we allow the verifier to

access “private” randomness that is not accessible to the prover.

Both the verifier and prover algorithms will have access to the input of the problem instance. The two algorithms will exchange messages sequentially, computing the next message in the sequence as a function of the messages up to that point. Ultimately, the verifier algorithm will decide whether to accept or reject the problem instance. We can think of the interaction metaphorically as the prover trying to “convince” the verifier of the problem instance being true, and of the verifier trying to verify that the prover is not “dishonest” or “cheating” and misleading the verifier into accepting a false statement.

Definition 2 (Interactive Proof system (IP)). An interactive proof system for a language L consists of an interactive PPT verifier algorithm $\mathcal{V}$ and an interactive (possibly inefficient) algorithm $\mathcal{P}$, which exchange a series of messages $m_1, \dots, m_k$, with each message computed as a function of all the previous messages: $m_i = \mathcal{V}(x, m_1, \dots, m_{i-1})$, and likewise for $\mathcal{P}$. Notably, the verifier’s computations may also depend upon private random bits not revealed to the prover. Denote by $(\mathcal{P}, \mathcal{V}(r))(x) = 1$ the event that the verifier $\mathcal{V}$, with private randomness r , accepts the interactive proof after communicating with the prover $\mathcal{P}$ on the joint input x and assuming $\mathcal{V}$ has randomness r . The following two properties are required to hold:

1. *Completeness*: $\forall x \in L$,

$$\Pr[(\mathcal{P}, \mathcal{V}(r))(x) = 1] \geq \frac{2}{3}$$

2. *Soundness*: $\forall x \notin L$ and $\forall$ (malicious and possibly all powerful) $\mathcal{P}^*$,

$$\Pr[(\mathcal{P}^*, \mathcal{V}(r))(x) = 1] \leq \frac{1}{3}.$$

Remark. These numbers ($\frac{2}{3}$ and $\frac{1}{3}$) are arbitrary. By repeating the interactive proof λ times and accepting if and only if at least $\frac{\lambda}{2}$ are accepting we can get completeness $1 - \text{negl}(\lambda)$ and soundness $\text{negl}(\lambda)$. This follows from the Chernoff bound, which is a concentration bound that says that if $X_1, \dots, X_\lambda$ are independent and identically distributed Bernoulli random variables such that $\Pr[X_i = 1] = p$ then $\Pr[|\frac{1}{\lambda} \sum_{i \in \lambda} X_i - p| > \delta] \leq 2^{-O(\delta^2 \cdot p \cdot \lambda)}$.

We therefore often give $\mathcal{P}$ and $\mathcal{V}$ an additional input 1^λ , and require completeness to hold with probability $1 - \text{negl}(\lambda)$ and soundness to hold with probability $\text{negl}(\lambda)$.

The class IP is the set of all languages L that have such an interactive proof. Note that $\text{NP} \subseteq \text{IP}$ but IP may contain additional languages.

See [this](#) for information about the Chernoff bound.

Remark. The power of the class IP comes from the fact that the verifier is randomized. If the verifier was deterministic this class would be equivalent to NP

Example: Graph Non-Isomorphism

A graph G is defined by a set of vertices V and a set of edges $E \subseteq V \times V$, where each edge is a pair (u, v) for $u, v \in V$. Two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are said to be isomorphic if there exists a permutation $\pi : V_1 \rightarrow V_2$ such that for every $u, v \in V_1$,

$$(u, v) \in E_1 \text{ if and only if } (\pi(u), \pi(v)) \in E_2.$$

Denote by

$$L_{\text{iso}} = \{(G_1, G_2) : G_1, G_2 \text{ are isomorphic}\}.$$

Note that $L_{\text{iso}} \in \text{NP}$. But we believe that the complement language:

$$\bar{L}_{\text{iso}} = \{(G_1, G_2) : G_1, G_2 \text{ are not isomorphic}\}$$

is not in NP.

Yet there is a simple interactive proof for proving membership in $\bar{L}_{\text{iso}}$. Given a pair of non-isomorphic graphs (G_1, G_2) the interactive proof proceeds as follows: In what follows, assume for simplicity that $V_1 = V_2 = [n]$. This is without loss of generality since we can simply rename the variables to be in $[n]$ assuming $|V_1| = |V_2| = n$.¹

1. V chooses a random $b \in \{1, 2\}$ and chooses a random permutation $\pi : [n] \rightarrow [n]$. It sends $\pi(G_b)$ to P .
2. P sends b' such that $G_{b'}$ is isomorphic to $\pi(G_b)$.
3. The verifier accepts if and only if $b' = b$.

This shows that interactive proofs are likely to be more powerful than NP! There is something else really nice about this proof: the verifier didn't learn anything about why the graphs are non-isomorphic! He knew that $b' = b$!

Definition of Zero-Knowledge

Definition 3 (Honest verifier ZK). Fix a language $L \in \text{NP}$ with a corresponding NP relation R . We say that an interactive proof $(\mathcal{P}, \mathcal{V})$ for L is *honest-verifier (computational) zero-knowledge* if there exists a PPT simulator $\mathcal{S}$ such that for every polynomials $\ell_1 = \ell_1(\lambda)$ and $\ell_2 = \ell_2(\lambda)$, the following holds:

Formally, we have defined a directed graph, because we have denoted an edge as an ordered pair. For this lecture this distinction is not important and you can think of the edges as non-directed.

¹ If $|V_1| \neq |V_2|$ then the verifier immediately knows that the two graphs are not isomorphic.

For every $\lambda \in \mathbb{N}$ every $x \in L$ of size $|x| = \ell_1(\lambda)$, every witness w with $(x, w) \in R$, and every auxiliary input $z \in \{0, 1\}^{\ell_2(\lambda)}$, the following ensembles (over the security parameter) are computationally indistinguishable:

$$\left\{ \text{View}\left((\mathcal{P}(w), \mathcal{V})(1^\lambda, x, z)\right) \right\}_{\lambda \in \mathbb{N}} \approx \left\{ S(1^\lambda, x, z) \right\}_{\lambda \in \mathbb{N}}.$$

In addition, it is required that $\mathcal{P}$ runs in polynomial time (given a witness w).

Definition 4 (ZK). Fix a language $L \in \text{NP}$ with a corresponding NP relation R . We say that an interactive proof $(\mathcal{P}, \mathcal{V})$ for L is (*computational*) *zero-knowledge* if for every PPT verifier $\mathcal{V}^*$ there exists a PPT simulator $\mathcal{S}$ such that for every polynomials $\ell_1 = \ell_1(\lambda)$ and $\ell_2 = \ell_2(\lambda)$, the following holds:

For every $\lambda \in \mathbb{N}$ every $x \in L$ of size $|x| = \ell_1(\lambda)$, every witness w with $(x, w) \in R$, and every auxiliary input $z \in \{0, 1\}^{\ell_2(\lambda)}$, the following ensembles (over the security parameter) are computationally indistinguishable:

$$\left\{ \text{View}\left((\mathcal{P}(w), \mathcal{V}^*)(1^\lambda, x, z)\right) \right\}_{\lambda \in \mathbb{N}} \approx \left\{ S(1^\lambda, x, z) \right\}_{\lambda \in \mathbb{N}}.$$

In addition, it is required that $\mathcal{P}$ runs in polynomial time (given a witness w).

Construction of ZK Protocol for the NP Language QR

Goldwasser Micali and Rackoff [2] showed how to construct ZK proof for some NP languages. For example, they constructed a ZK proof for the language

$$\text{QR} = \{(N, y) : \exists x \in \mathbb{Z}_N^* \text{ s.t. } y = x^2 \bmod N\}$$

The proof is very simple! Given an instance (N, y) it proceeds as follows:

1. The prover chooses a random $r \leftarrow \mathbb{Z}_N^*$ and sends to the verifier the element $s = r^2 \bmod N$.
2. The verifier chooses a random bit $b \leftarrow \{0, 1\}$.
3. If $b = 0$ the prover sends $z = r$. If $b = 1$ the prover sends $z = r \cdot x$, where x is the witness (i.e., $y = x^2 \bmod N$).
4. If $b = 0$ the verifier accepts iff $z^2 = s \bmod N$. If $b = 1$ the verifier accepts iff $z^2 = s \cdot y \bmod N$.

This protocol is ZK. Let us first prove that it is honest-verifier ZK. The simulator will choose a random bit $b \leftarrow \{0,1\}$ on behalf of the verifier. If $b = 0$ it will first choose the prover's first message, by sampling a random $r \leftarrow \mathbb{Z}_N^*$ and setting $s = r^2 \bmod N$, and then set the prover's second message to be $z = r$. If $b = 1$ then it will first choose the prover's second message, by sampling a random element $z \leftarrow \mathbb{Z}_N^*$, and then set the prover's first message to be $s = z^2 \cdot y^{-1} \bmod N$.

This protocol is also (malicious verifier) ZK. The simulator is very similar to the one above. It will guess the (malicious) verifier's bit $b \leftarrow \{0,1\}$, and produce (s,z) as above. If $\mathcal{V}(x,s) = b$ then output (s,b,z) as the simulated transcript. Else, try again.

This protocol has perfect completeness and soundness $1/2$. To amplify the soundness we simply repeat the entire protocol.

Remark. To maintain the zero-knowledge property we need to repeat the protocol *sequentially*. In general parallel repetition does not preserve ZK (see HW 4). But sequential repetition does!

If all we want is honest-verifier ZK then we can repeat the protocol in parallel.

Constructing Zero-Knowledge Proofs for all of NP

Goldreich Micali and Wigderson proved that every language in NP has a zero-knowledge proof, assuming one-way functions exist [1]. In other words, every proof can be made zero-knowledge!

We will see a zero-knowledge proof for a specific NP-complete language called 3Col which contains the set of all graphs $G = (V, E)$ such that the set of vertices V can be colored by three colors: $C : V \rightarrow \{1,2,3\}$ such that no two adjacent vertices have the same color. Namely, for every $(u,v) \in E$, $C(u) \neq C(v)$.

We convert a coloring $C : V \rightarrow \{1,2,3\}$, which is a proof that the graph G is 3-colorable, into a zero-knowledge proof, as follows:

1. The prover does the following:
 - (a) Choose a random permutation $\pi : \{1,2,3\} \rightarrow \{1,2,3\}$.
Denote by $V = \{1,2,\dots,n\}$.
 - (b) For every $i \in [n]$ place the color $\pi(C(i))$ in an opaque locked box and send the n locked boxes to the verifier.
2. The verifier chooses a random edge $(i,j) \in E$ and sends (i,j) to the prover.
3. The prover sends the keys that open only box i and box j .

4. The verifier accepts if and only if the colors in these boxes are distinct and are legal (i.e., belong to the set $\{1, 2, 3\}$).

Is this protocol zero-knowledge? It is definitely honest-verifier zero-knowledge since the only thing the verifier learned is two distinct random colors. We can simulate this by choosing a random edge (u, v) (on behalf of the verifier), storing two random distinct colors $c_u, c_v \in \{1, 2, 3\}$ and storing these colors in the boxes corresponding to vertices u and v , and then opening these boxes. This view is identical to the real view.

It is also (malicious verifier) zero-knowledge. The simulator can guess the verifier's edge (u, v) randomly, store the random distinct colors $c_u, c_v \in \{1, 2, 3\}$ in their boxes, give the boxes to the malicious verifier, and if the verifier sends back the edge (u, v) the continue the simulation. If not, try again!

It has completeness 1. The soundness is only $1 - \frac{1}{|E|}$ but can be amplified via repetitions. Each time we repeat we need to choose a fresh permutation.

Remark. The above protocol is a physical protocol, where the prover sends opaque locked boxes. Such boxes have a digital analogue. This is called a commitment scheme.

Currently, we think of the boxes as ideal opaque physical boxes. But soon we will replace them with a digital commitment to each color, and a malicious verifier can choose its edge as a function of this commitment string.

Commitment Schemes

Definition 5. A commitment scheme corresponding to a message space $\mathcal{M}$ consists of a pair of algorithms (Gen, Com) :

- **Gen** is a PPT algorithm that takes as input the security parameter 1^λ and outputs public parameters, denoted by $\text{pp} \in \{0, 1\}^{\text{poly}(\lambda)}$.
- **Com** is a polynomial-time computable function that takes an input public parameters pp , a message $m \in \mathcal{M}$ and randomness $r \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$ and outputs a commitment $\text{Com}(\text{pp}, m, r)$.

The following two properties are required to hold:

- **Hiding:** For every $m_0, m_1 \in \mathcal{M}$,

$$(\text{pp}, \text{Com}(\text{pp}, m_0, r_0)) \approx (\text{pp}, \text{Com}(\text{pp}, m_1, r_1))$$

for $\text{pp} \leftarrow \text{Gen}(1^\lambda)$ and $r_0, r_1 \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$.

- **Binding:** For every (even all-powerful) adversary $\mathcal{A}$:

$$\Pr_{\text{pp} \leftarrow \text{Gen}(1^\lambda)} [A(\text{pp}) = (m_0, r_0, m_1, r_1) \text{ s.t. } m_0 \neq m_1 \wedge \text{Com}(\text{pp}, m_0, r_0) = \text{Com}(\text{pp}, m_1, r_1)] = \text{negl}(\lambda).$$

One can think of both **Gen** and **Com** as randomized algorithms. We chose to explicitly include the randomness of **Com**, and hence think of it as being deterministic since when the commitment is opened the randomness is revealed

Constructing a Commitment Scheme

It is known how to construct a commitment scheme from the minimal assumption that one-way functions exist. We will see a simple construction from injective one-way functions.

Fix an injective one-way function $f : \{0,1\}^* \rightarrow \{0,1\}^*$. Let P be a (randomized) hardcore predicate corresponding to f . Define

$$\text{Com}(b, r, s) = (f(r), s, P(r, s) \oplus b)$$

Note that this scheme has no public parameters. Computational hiding follows from the fact that P is a hardcore predicate of f , which by definition implies that

$$(f(r), s, P(r, s)) \approx (f(r), s, U)$$

where $U \leftarrow \{0,1\}$ is a uniformly random bit.

Binding follows from the fact that f is injective.

References

- [1] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to prove all np-statements in zero-knowledge, and a methodology of cryptographic protocol design. In Andrew M. Odlyzko, editor, *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings*, volume 263 of *Lecture Notes in Computer Science*, pages 171–185. Springer, 1986.
- [2] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In Robert Sedgewick, editor, *Proceedings of the 17th Annual ACM Symposium on Theory of Computing, May 6-8, 1985, Providence, Rhode Island, USA*, pages 291–304. ACM, 1985.